# SLOWKING: Offline Belief-State Search for a Closed, Simultaneous-Move Battle Engine

**Learning a Tier-3 game solver for Pokémon Champions from logged replays alone**

**Version 1.0 · 2026-07-23 · Will Hooper · ABRA**

> *The definitive design for ABRA's deepest model. It states a problem that, in this exact combination, is not solved in the published literature — equilibrium-aware search in a* ***closed****,* ***simultaneous-move****,* ***imperfect-information*** *commercial game engine that we may only observe* ***offline*** *— derives the algorithm end to end, grounds every component in the 2020–2025 record (ReBeL, Student of Games, PokéChamp, Metamon, MuZero/Gumbel, CFR, offline RL), and gives an honest, staged feasibility plan built on data ABRA already collects. Living document; updated in the same pass as the code it describes. Companion: research roadmap, simulator white paper.*

## Abstract

ABRA collects a growing corpus of Pokémon Champions (Reg M-B) replays — a two-player, zero-sum, *simultaneous-move*, imperfect-information stochastic game whose transition dynamics are implemented by a **closed engine we cannot query**. SLOWKING is the model that plays this game near-optimally: it searches over **belief states** — distributions over what the opponent is hiding — to return an equilibrium-aware policy rather than a greedy best move. The published state of the art for imperfect-information search (ReBeL, Student of Games, Player of Games) assumes either a *known* simulator or the ability to interact with the environment; the strongest Pokémon agents (PokéChamp, Metamon) either wrap a *reconstructed open-source* engine or learn model-free from logs without equilibrium search. SLOWKING targets the intersection none of them occupy: **equilibrium search over a learned model of a closed engine, trained purely offline, in a simultaneous-move doubles setting.** We contribute (i) a formalisation of Champions as a partially observable stochastic game with a public belief state; (ii) a **grey-box world model** that learns only the *residual* over CHOMP's exact damage calculator, shrinking the learning problem to the parts CHOMP cannot express (status chains, secondary effects, switch dynamics, position); (iii) a belief representation and offline update mined from revealed-set statistics; (iv) a value/policy pair trained offline with conservative RL and warm-started by a behaviour-clone of real ladder play; and (v) a tractable belief-state search that solves each simultaneous-move subgame to a *mixed* strategy and prunes the combinatorial action space with a Gumbel policy-improvement operator. We specify an evaluation protocol centred on **exploitability**, not just win rate, and give a staged build plan whose early stages are startable today on ABRA's data. We do not claim SLOWKING is trained; we claim the problem is well-posed, the design is sound, and the foundation is in place.

## 1. Introduction

A competitive VGC player answers three nested questions: which six to build (DITTO), which four to bring (CHOMP), and *how to play the resulting game*. The third is the hardest and the highest-ceiling, and it is where every shallow model in ABRA stops. JOLTEON (Tier 1) scores teams in one forward pass but has no notion of a turn. MEDICHAM (Tier 2) plays a matchup out with exact damage and a behaviour-cloned policy, but it is a *fixed-policy rollout*: it never asks "what is my opponent's best response to this line, and what should I mix to be unexploitable?" SLOWKING is the model that asks that question and answers it with search.

Three properties make this genuinely hard, and their *conjunction* is what places SLOWKING outside the existing literature:

1. **The engine — assumed closed, likely open (v1.1 correction).** This paper was written on the assumption that Champions is a *closed* engine we could not query, forcing us to *learn* its dynamics from logged replays. **On investigation that assumption appears to be wrong.** The Champions format ( `[Gen 9 Champions] VGC 2026 Reg M-A/M-B` ) is defined in the **open-source** `smogon/pokemon-showdown` **repository** ( `config/formats.ts` ), and the SP stat system is a documented mod — i.e. it can very likely be run as a local, queryable simulator, exactly like the standard formats poke-env exposes to RL agents. **If confirmed runnable end-to-end, this removes the single hardest component of SLOWKING** (learning a world model, §4.1/§5): we would instead *query the real engine* for any hypothetical state, turning the problem from "offline model learning of a closed engine" into "equilibrium search over a **known** simulator" — the exact setting ReBeL and AlphaZero operate in, and far more feasible. It also unlocks **self-play** and a ground-truth engine for MEDICHAM, CHOMP, DITTO, and JOLTEON. The one open item is verifying the SP mod runs on `master` (not a private branch); the format config is confirmed public. The rest of this paper's learned-dynamics track (§4.1, §5) is retained as the fallback for the case where the local build proves incomplete, but the **known-simulator path is now the primary plan.**
2. **Moves are simultaneous.** Both players commit each turn without seeing the other's choice. The per-turn subgame is therefore a matrix game whose optimal solution is generally a *mixed* Nash equilibrium. A searcher that assumes a deterministic opponent is exploitable exactly at the decision points that matter.
3. **Information is imperfect, and we are offline.** We see the opponent's six at preview and each set only as it is revealed. The relevant object is a belief over hidden information, and we must learn to track and search over it *without ever interacting with the real engine* — only from logged trajectories, with all the distribution-shift hazards offline RL is heir to.

The rest of this paper formalises the game (§2), explains precisely why the standard playbook does not port unmodified (§3), derives SLOWKING component by component (§4), specifies offline training (§5) and the tractability machinery (§6), gives an honest evaluation protocol (§7) and feasibility assessment (§8), situates the work against prior art (§9), and states what is genuinely novel (§10).

## 2. The game, formally

A Champions battle is a **partially observable stochastic game (POSG)**: a finite set of players `i ∈ {1,2}` , a state space `S` , per-player action sets, a stochastic transition kernel, a zero-sum terminal reward, and per-player observations.

- **State** `s ∈ S` comprises both teams' species, full sets (moves, item, ability, the Champions SP spread), current HP, status, stat stages, field conditions (weather, terrain, screens, Trick Room, Tailwind timers), and which Pokémon are active. Level 50; no Tera.
- **Actions.** At each turn player `i` chooses *simultaneously* from `A_i(s)` : for each of two active Pokémon, a move (with target) or a switch. A move may be accompanied by a **Mega Evolution** — a once-per-battle, per-side step that must be modelled as part of the action: it changes the Pokémon's forme, base stats, typing, and ability for the remainder of the game (and shifts speed order the turn it happens), so `A_i(s)` includes a mega flag on eligible attackers, and the transition applies the forme change before damage. The joint action is `a = (a_1, a_2)` . Team preview is a special first move: an ordered bring of four from six, `|A_i| = 6·5·4·3 = 360` .
- **Transition** `T(s' | s, a)` is stochastic — damage rolls (16 outcomes, 85–100%), accuracy, secondary-effect procs, critical hits, speed ties. **This kernel is the closed engine; reproducing it is the crux.**
- **Reward** is terminal and zero-sum: `+1` win, `-1` loss.

- **Observation** `o_i = O_i(s)` : own team fully; of the opponent, the six species at preview and each move/item/ability only once used or triggered.

Because information is imperfect, the decision-relevant object is not the world state `s` but the **information state** — the history of one player's own actions and observations — and, for search, the **public belief state (PBS)**: the common-knowledge distribution over the pair of information states, given the public history. Following ReBeL, converting the game to a game *over public belief states* makes an imperfect-information game behave, for the purposes of value-function search, like a perfect-information one: the value of a PBS is well-defined and learnable, and search can proceed on PBS nodes. This PBS reformulation is the theoretical backbone of §4.4.

Two structural facts govern everything downstream: **simultaneous moves ⇒ mixed strategies** (each turn is a matrix game; the solution is a distribution), and **imperfect information ⇒ belief states** (we plan over `b`, not `s`). The team-building layer above the battle is a **Bayesian game** — commit a team before seeing the opponent's — and is DITTO's problem, evaluated by the battle values SLOWKING defines.

## 3. Why the standard playbook does not port unmodified

The modern recipe for superhuman imperfect-information play — ReBeL and Student of Games — is **self-play RL + search over public belief states, with a learned value/policy and a CFR-style subgame solver.** It is proven to converge toward Nash in two-player zero-sum games and reaches superhuman poker and strong Scotland Yard / chess / Go. We adopt its skeleton. But three assumptions it leans on are violated here, and each violation forces a design change:

- **It assumes a known, queryable simulator (or environment access) for self-play.** ReBeL and SoG generate their own trajectories by playing the game forward through a known ruleset. We *cannot*: the Champions engine is closed. **Consequence:** we must first *learn* the transition model `Tθ` (§4.1) and do all training offline (§5). This is the single biggest departure and the reason SLOWKING is a research effort rather than a re-implementation.
- **Canonical treatments are turn-based (sequential).** Poker and Scotland Yard reveal actions in order; the PBS machinery is usually derived for sequential moves. Champions is *simultaneous*. **Consequence:** the subgame at each node is a genuine matrix game, and the solver must return a mixed strategy over the joint action (§4.5), not a sequence of single-agent decisions.
- **Offline RL is fragile.** Learning a value/policy purely from logged data invites distribution-shift and value over-estimation on actions the data never took. **Consequence:** we use *conservative* offline RL (CQL/IQL-style pessimism) and anchor the policy to a **behaviour-clone** of real play so search never wanders far from supported states (§4.3, §6).

PokéChamp and Metamon confirm the domain is tractable but sit on the other side of these constraints: PokéChamp runs a minimax LLM agent over the *open-source* engine with no equilibrium guarantee and no learned dynamics; Metamon trains strong *model-free* offline-RL agents from 5M+ reconstructed Showdown trajectories but does no belief-state search. SLOWKING = the belief-state searcher, over a *learned* model of a *closed* engine, in the *simultaneous-move* setting — the box none of them tick.

## 4. The SLOWKING architecture

SLOWKING has five components: a grey-box world model, a belief tracker, an offline value/policy pair, a belief-state search that ties them together, and a per-turn mixed-strategy subgame solver.

### 4.1 Grey-box dynamics `Tθ(s' | s, a)` — learn only the residual over CHOMP

The naïve approach learns the entire transition kernel from scratch. That is wasteful: we already own an *exact* partial simulator of the dominant effect. CHOMP's `champ-model.js` computes, deterministically and correctly, the damage distribution of any move in any matchup — STAB, type chart, weather, items, abilities, spread, the full 16-roll range — validated against the reference calculator. So we factor the kernel:

```
T(s' | s, a)  =  T_known(s' | s, a; CHOMP)  ⊙  Tθ_resid(s' | s, a)
```

`T_known` applies CHOMP's exact damage and the mechanically certain effects (fainting, weather setting, speed order under known modifiers). `Tθ_resid` — the only learned part — models what CHOMP does not express: status infliction and its downstream (sleep turns, burn/paralysis dynamics), secondary-effect probabilities, switch decisions' consequences, redirection and position in doubles, and the residual between predicted and observed damage. This is **grey-box system identification**, and it shrinks the learning target to a small, well-supported delta. Two properties of ABRA's data make it learnable now: we store the **per-turn stream** (move order → observed speed, exact damage % per move) for 30k+ turns, and the **behaviour-clone** ( `data/move-priors.json` ) already captures the empirical action policy the dynamics must be consistent with. The model is calibrated against `data/dynamics.json` 's observed per-move damage distributions — MEDICHAM's grounding becomes SLOWKING's supervision. *Literature:* MuZero and Dreamer show learned models suffice for planning; Metamon shows Showdown dynamics are learnable at scale from logs.

### 4.2 Belief representation and offline update

The belief `b` is a distribution over the opponent's hidden variables: which four of their six were brought, and each brought Pokémon's full set (moves/item/ability/spread) consistent with what has been revealed. We represent `b` as a **factored particle set**: a bag of concrete opponent configurations, each weighted, sampled from priors and reweighted by observations. The priors are *learned from the corpus, not assumed*: ABRA's usage model gives bring/lead rates; the behaviour-clone gives per-species set and move priors; revealed moves/items hard-constrain the support (a Pokémon that has clicked Protect and Moonblast cannot be a four-attacks set). The update is Bayesian filtering: on each observation `o` , reweight each particle by `P(o | particle)` under `Tθ` , resample, and prune particles the observation has falsified. Because every prior is mined from stored data, belief tracking is itself an *offline-trained* component — no environment access required.

### 4.3 Value and policy from offline data

Search needs a **value function** `V(b)` (the equilibrium value of a public belief state) and a **policy prior** `π(a | b)` (a distribution to guide and prune search). We obtain both offline:

- **Policy prior** is initialised from the behaviour-clone — the empirical action distribution of real ladder players, per species and turn context ( `policy.js` ), already built. This is a strong, data-supported prior that keeps search inside the manifold of plays the data covers, directly mitigating offline distribution shift.
- **Value** is trained with **conservative offline RL** (CQL/IQL-style): learn `V` from logged returns while penalising optimism on out-of-distribution actions, so the searcher is not seduced by states the data never validated. The value target is bootstrapped through the learned model and regressed toward realised game outcomes — the reward is terminal and known (win/loss), which is a gift for credit assignment.

Both are refined by search via a policy-improvement loop (§6): the searched policy at a state becomes a supervised target for `π` , and the searched value becomes a target for `V` — the ReBeL/SoG self-improvement loop, run **offline through the learned model** instead of the real engine.

### 4.4 Belief-state search

At decision time SLOWKING builds a search tree whose nodes are **public belief states**. From the current PBS it expands a few plies: at each node it (a) samples opponent configurations from the belief (§4.2), (b) proposes candidate joint actions from the policy prior (§4.3), (c) rolls them through the grey-box model `Tθ` (§4.1) to child PBSs, and (d) backs up values from the learned `V` at the leaves. The novelty over perfect-information MCTS is that expansion and back-up operate on *belief* nodes and that each internal node is solved as a *simultaneous-move subgame* (§4.5) rather than a max/min. This is exactly the ReBeL construction — value-function search over PBS with a CFR-style equilibrium solver at subgames — adapted to (i) a learned model in place of a known one and (ii) simultaneous rather than sequential moves. *Literature:* ReBeL (PBS value search + CFR), Student of Games / Player of Games (growing-tree CFR with learned value, sound convergence).

### 4.5 The simultaneous-move subgame solver

Each turn is a matrix game: rows are player 1's legal joint actions (per-slot move/switch), columns are player 2's, and the payoff of a cell is the searched value of the resulting child PBS. Its solution is a **mixed Nash equilibrium** — a distribution over actions for each side. We compute it with **regret-matching / CFR** iterated to low exploitability inside the subgame, yielding `σ_i(a_i | b)`. SLOWKING then plays its own mixed strategy `σ_1`, which is unexploitable *within the searched horizon* by construction. Two practical points: the combinatorial action space (two slots, moves × targets × switches) is pruned to the top candidates by the policy prior before the matrix is formed (§6), and the subgame payoffs reuse the same `Tθ` / `V` as the outer search, so the solver adds equilibrium reasoning without a second model.

## 5. Offline training pipeline

Every component trains from the durable store, never from the live engine, and improves as the store grows — the flywheel, applied to Tier 3:

1. **Trajectories.** Convert each stored replay into `(s, o, a, r)` transitions via a game-spec encoder (`engine/game-spec.js`, the roadmap's Paper 1 — built as the first concrete step; see §8). Champions' per-turn stream + raw-log archive means this is a re-parse, never a re-pull.
2. **Grey-box residual** `Tθ` is fit to observed transitions with CHOMP's `T_known` subtracted out, supervised by observed damage (`dynamics.json`) and move order.
3. **Belief priors** are the existing usage model + behaviour-clone; no training beyond what ABRA already computes.
4. **Value/policy** are trained with conservative offline RL, warm-started by the behaviour-clone, then refined by searched targets (§4.3).
5. **Search-and-distil** closes the loop: run belief-state search over `Tθ` on sampled states, distil the searched policy/value back into `π` / `V`, repeat. This is self-improvement without self-play against a real engine — the offline analogue of ReBeL's training loop.

## 6. Making it tractable

Three levers keep the cost bounded. **(a) Policy-guided pruning:** the behaviour-cloned prior collapses each turn's enormous action set to a handful of plausible candidates before any search — you do not simulate every path; the policy proposes the few viable ones (the AlphaStar/Gumbel insight). **(b) Gumbel policy improvement:** at the root we select actions by sampling-without-replacement with the Gumbel-Top-k trick, which guarantees a policy-improvement step even with *very few* simulations — Gumbel MuZero learns reliably at as few as two simulations per move, the property that makes second-scale planning feasible. **(c)**

**Coarse-to-fine across tiers:** SLOWKING is only ever invoked on the handful of states that matter — JOLTEON screens thousands of teams, MEDICHAM re-ranks finalists, and SLOWKING vets the last few or the critical in-game turns. No expensive belief search is spent where a cheap model suffices.

## 7. Evaluation protocol

Win rate alone is misleading in a high-variance game, and a searcher can win yet be exploitable. We evaluate on four axes:

1. **Head-to-head** vs the ladder of baselines already in ABRA — the behaviour-clone policy, MEDICHAM's fixed-policy rollout, and a greedy-damage agent — under identical team samples, reporting win rate with confidence intervals.
2. **Exploitability proxy.** Against a best-response opponent computed within the learned model, measure how much the opponent can gain — the operational definition of "how close to equilibrium." A model-free agent has no such guarantee; this is where belief search should distinguish itself.
3. **Calibration.** Brier score and reliability curves on SLOWKING's win-probability estimates vs realised outcomes on held-out games (temporal split), as we already do for JOLTEON.
4. **Ablations.** Grey-box vs learn-from-scratch dynamics; behaviour-clone prior vs uniform; mixed-Nash subgame solver vs deterministic max — each isolates a claimed contribution.

Crucially, results are reported against the domain's irreducible variance (§ the simulator paper's honesty about a ~55–57% team-only ceiling): SLOWKING's job is to raise the *in-game* decision quality above MEDICHAM's fixed policy and to be *unexploitable*, not to manufacture certainty the game does not contain.

## 8. Feasibility, staged and honest

We do not claim SLOWKING is built. We claim it is well-posed and that the on-ramp exists:

- **Ready today.** The offline corpus (30k+ turn transitions, growing hourly), the exact damage prior (CHOMP), the observed-dynamics supervision ( `dynamics.json` ), and the behaviour-clone policy prior ( `move-priors.json` ) are all built and tested. Two of the five roadmap papers (the game-spec/data and the value/policy) are startable now on this data.
- **First concrete step (this release):** `engine/game-spec.js` — the state/observation/action encoding and a converter that turns stored replays into `(s, o, a, r)` trajectories. This is Paper 1's deliverable and it makes SLOWKING a build with a first commit, not only a design.
- **The research.** The grey-box residual model, the belief-state search, and the offline search-and-distil loop are genuine research — the frontier ReBeL/SoG/Metamon occupy. They need real compute, careful offline-RL practice, and sustained work, and each should be judged against the MEDICHAM baseline before its cost is accepted. Recent results prove this class of method *works*; none prove it is cheap.

The disciplined path: ship the data encoder and value/policy on current data; stand up the grey-box model against observed dynamics; add belief-state search last, gated by exploitability against MEDICHAM. Every stage is useful on its own, and every stage feeds the models above it.

## 9. Relationship to prior work

- **ReBeL** (Brown et al., NeurIPS 2020) — RL+search over public belief states with a CFR subgame solver; converges toward Nash in 2p0s games; superhuman poker. SLOWKING's search skeleton, adapted to a learned model and simultaneous moves.
- **Student of Games** (Schmid et al., Science Advances 2023) and **Player of Games** (Schmid et al., 2021) — a unified sound algorithm across perfect/imperfect information via growing-tree CFR with learned

value; strong in chess, Go, poker, Scotland Yard. The template for combining search, learning, and game-theoretic reasoning.

- **MuZero** (Schrittwieser et al., Nature 2020) and **Gumbel MuZero / Policy improvement by planning with Gumbel** (Danihelka et al., ICLR 2022) — planning with a learned model; policy improvement with few simulations. SLOWKING's learned-model and low-simulation planning basis.
- **PokéChamp** (Karten et al., ICML 2025) — expert-level minimax *LLM* agent over the open-source engine (~1500 Elo, 1M+ game dataset), no learned dynamics, no equilibrium guarantee. A domain proof and a baseline, on the other side of the closed-engine and equilibrium constraints.
- **Metamon** (UT Austin RPL, RLC 2025) — human-level *model-free* offline RL from 5M+ reconstructed Showdown trajectories. Proves offline learning of Showdown play at scale; no belief search.
- **VGC-Bench** (Angliss et al., 2025) — a doubles-VGC benchmark with PSRO/self-play baselines; the double-oracle machinery DITTO's outer loop mirrors.
- **CFR** (Zinkevich et al., 2007) and **offline RL** — CQL (Kumar et al., 2020), IQL (Kostrikov et al., 2021), Decision Transformer (Chen et al., 2021) — the equilibrium and offline-learning foundations.

## 10. What is genuinely new

Each ingredient exists; their *combination*, in this domain, does not appear in the literature:

> ***Equilibrium-aware belief-state search over a learned grey-box model of a closed battle engine, trained entirely offline from public replays, warm-started by a behaviour-clone mined from the same corpus, solving simultaneous-move subgames to mixed strategies, in doubles VGC.***

ReBeL/SoG assume a known model or environment access; PokéChamp uses an open engine and no equilibrium search; Metamon is model-free with no search; MuZero/Gumbel are perfect-information or single-agent. The grey-box factorisation that learns only the residual over an exact damage calculator (CHOMP) is, to our knowledge, a new device for making offline model-learning tractable in this setting, and it is only possible because ABRA already owns both the exact partial simulator and the observed-dynamics supervision to fit the residual against. That is the paper's thesis: **the closed engine is not a wall but a residual, and the corpus we already store is enough to learn it.**

## 11. References


1. Brown, N., Bakhtin, A., Lerer, A., Gong, Q. (2020). *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games (ReBeL)*. NeurIPS. arXiv:2007.13544.
2. Schmid, M. et al. (2023). *Student of Games: A unified learning algorithm for both perfect and imperfect information games*. Science Advances 9(46). DOI:10.1126/sciadv.adg3256.
3. Schmid, M. et al. (2021). *Player of Games*. arXiv:2112.03178.
4. Schrittwieser, J. et al. (2020). *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero)*. Nature 588.
5. Danihelka, I., Guez, A., Schrittwieser, J., Silver, D. (2022). *Policy improvement by planning with Gumbel*. ICLR.
6. Karten, S., Nguyen, A. L., Jin, C. (2025). *PokéChamp: an Expert-level Minimax Language Agent for Competitive Pokémon*. ICML (spotlight). arXiv:2503.04094.
7. UT Austin RPL (2025). *Metamon: Human-Level Competitive Pokémon via Scalable Offline RL with Transformers*. RLC. arXiv:2504.04395.
8. Angliss, C., Cui, J., Hu, J., Rahman, A., Stone, P. (2025). *VGC-Bench: Towards Mastering Diverse Team Strategies in Competitive Pokémon*. AAMAS 2026. arXiv:2506.10326. (Title corrected 2026-09-09 against the arXiv record; this entry had carried a different subtitle.)

9. Zinkevich, M., Johanson, M., Bowling, M., Piccione, C. (2007). *Regret Minimization in Games with Incomplete Information (CFR)*. NIPS.
10. Kumar, A., Zhou, A., Tucker, G., Levine, S. (2020). *Conservative Q-Learning for Offline RL*. NeurIPS.
11. Kostrikov, I., Nair, A., Levine, S. (2021). *Offline RL with Implicit Q-Learning (IQL)*.
12. Chen, L. et al. (2021). *Decision Transformer: RL via Sequence Modeling*. NeurIPS.
13. Hafner, D. et al. (2023). *Mastering Diverse Domains through World Models (DreamerV3)*. arXiv:2301.04104.

---

**Companion documents.** Research roadmap · Simulator white paper · Cheat sheet · Executive summary